Risk-aware portfolio optimization

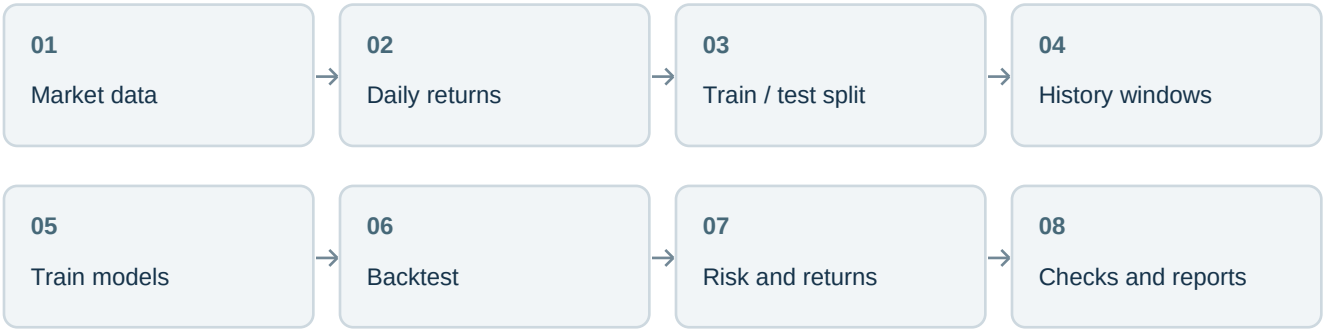
A plain-language guide to the project pipeline

This project is a research pipeline for testing ways to divide money among investments while balancing returns and risk. It compares ordinary machine-learning models with models that use simulated quantum circuits, checks whether two quantum implementations behave consistently, and studies how the resulting portfolios perform.

A portfolio is a collection of investments. Portfolio optimization means choosing how much of the total to allocate to each investment. For example, the system might propose 40% in one stock, 30% in another, 20% in a third, and 10% in a fourth.

Its central question is: Can a model learn useful allocation decisions from recent market history, and how well do those decisions hold up when tested?

This guide is based on a review of the main data, training, quantum-model, evaluation, and experiment code, along with supporting documentation and saved results. It describes the checkout reviewed on October 19, 2026. Training experiments were not rerun for this explanation.



Read left to right across each row. The second row continues the pipeline.

1. How the repository fits together

The repository contains three connected research efforts. Understanding these helps make sense of the many scripts.

The first is the original research implementation, in original_pennylane/qrl-dpo-public/. This provides the learning algorithms, original quantum model, data-processing utilities, and original experiment runner.

The second is pennylane_reproduction/. This asks whether the original code can reproduce its own results and how those results compare with the research paper. Reproducing an experiment means running it again under specified conditions and checking whether the outcome is consistent.

The third is qiskit_port/. This rebuilds the quantum model using a different software toolkit and adds benchmark experiments, risk measurements, reliability checks, and reporting tools. A port is an implementation adapted to another software system.

PennyLane and Qiskit are software toolkits for building and evaluating quantum calculations. They are not separate investment strategies. The project uses them to implement corresponding versions of the same underlying quantum model.

2. Start with historical market data

The model mainly works with daily returns, meaning daily percentage changes, rather than dollar prices themselves. If a stock goes from $100 to $102, its daily return is:

> **102 / 100 - 1 = 0.02, or 2%.**

Returns make different investments easier to compare: a $20 stock and a $500 stock can both rise by 2%.

There is an important distinction in this repository:

- The original file named price_data.parquet.gzip is treated by the code as already containing returns.
- The custom datasets contain prices, so the benchmark converts them into returns first.

Parquet and CSV are simply file formats for storing tables. They are not different kinds of financial data.

The custom-data downloader gets adjusted closing prices through Yahoo Finance using yfinance, a Python data-access library. Adjusted means the price history accounts for events such as stock splits and dividend adjustments.

One custom dataset contains NVDA, AMD, TSLA, AMZN, META, NFLX, COST, UNH, BA, and KO. These were selected to have no overlap with the original paper's investment list.

The data-loading code checks for missing or invalid numbers and warns about unusually large daily changes. Accidentally treating a dollar price as a percentage return would make the entire experiment meaningless.

Source: qiskit_port/benchmark_lib.py; qiskit_port/download_custom_nonpaper_10_data.py.

3. Separate learning from testing

The main comparison benchmarks divide dates in chronological order:

- The earliest 60% is training data: the material the model learns from.
- The next 20% is validation data: material used to monitor learning.
- The final 20% is test data: later dates used to measure the finished model.

For a benchmark containing 240 trading days, this becomes 144 training days, 48 validation days, and 48 test days.

A useful analogy is studying, taking practice exams, and then taking the final exam. The final exam should not become part of the study material.

Keeping dates in order matters. Randomly mixing old and new dates could let information from later periods influence a model that is supposed to act earlier.

The original research runner uses a different arrangement called expanding-window cross-validation. It repeats the experiment over several date ranges, increasing the amount of earlier history available for training. Each repetition is called a fold.

The original reproduction and the newer small benchmarks therefore do not use identical experimental setups.

4. Turn recent history into a model input

The model receives a short window of recent history, called its lookback window. With four investments and a ten-day lookback, the input contains:

> **4 investments x 10 days = 40 numbers.**

These numbers describe what happened recently. The code flattens the small table into one long list so the model can process it.

In the learning code, this list is called the state: the information available when making a decision. The corresponding action is the allocation the model proposes, such as [0.40, 0.30, 0.20, 0.10]. These are portfolio weights, meaning fractions of the portfolio.

The word weights has two meanings here:

- Portfolio weights determine where the money goes.
- Model weights are adjustable internal numbers that determine how the model calculates its answer.

The data loader creates pairs of consecutive historical windows for training. With forecasting disabled, these advance in lookback-sized blocks, rather than creating a new overlapping training example for every single day.

The original setup also supports an optional ARIMA forecast, a statistical method that extrapolates patterns in a time series. Predicted values can be appended to the historical input. The two main newer benchmarks set the forecast window to zero, so this extra forecasting step is disabled.

Source: original_pennylane/qrl-dpo-public/utilities/data_processing.py.

5. Meet the actor and critic

The learning models use a decision-maker and an evaluator, called the actor and the critic.

The actor asks: Given this recent market history, how should I allocate the portfolio? It receives the market state and produces portfolio weights.

The critic asks: Given this history and this proposed allocation, how valuable does that decision appear to be? It receives both the market state and the proposed weights, and produces one score.

That score is often called a Q-value. It is the critic's learned estimate of the usefulness of an action, including the future-value term used by the training rule.

This is reinforcement learning: learning to make decisions using numerical feedback about how desirable those decisions are.

The critic is itself learning. It does not start out as a trustworthy financial expert. The actor and critic improve together through repeated updates.

6. Where the risk awareness enters

The core training reward is:

> **reward = average portfolio return + risk preference x volatility**

Volatility measures how much returns fluctuate. A portfolio with large up-and-down movements has higher volatility. The benchmark risk-preference settings are negative, so volatility lowers the reward.

For illustration, if average daily return is 0.2%, volatility is 1%, and risk preference is -0.5:

> **reward = 0.2% - 0.5 x 1% = -0.3%.**

This encourages a tradeoff between return and fluctuations.

However, the immediate reward is calculated using the current historical window, not directly from the following window's realized return. The next window enters through the critic's estimated future value.

The training procedure is therefore more specific than making a trade, observing tomorrow's profit, and learning from it. The later backtest is where allocations are evaluated on subsequent dates.

Risk-aware here primarily refers to the volatility penalty in this reward. Trading costs and extreme-loss measurements are largely evaluated afterward; they are not all included in the training objective.

7. What happens during training

A training update follows a repeated feedback loop:

- 1. The actor proposes an allocation for a historical window.
- 2. The trainer adds random variation to explore alternative allocations.
- 3. It calculates the reward.
- 4. It stores the experience in a replay buffer, a collection of previous training examples.
- 5. It randomly selects a stored example to learn from.
- 6. It updates the critic so its score better matches the training target.
- 7. It updates the actor to produce allocations the critic scores more highly.

The critic's target combines the immediate reward with an estimate of future value. Gamma, also called the discount factor, controls how much that future estimate matters. Smaller values place less emphasis on the future-value term.

An epoch is one pass through the available training examples. A learning rate controls how large each adjustment to the model is.

A loss is the numerical quantity training tries to reduce. The critic's loss measures disagreement with its target. The actor's loss encourages a higher critic score, with optional penalties on its internal parameters.

Those penalties are called regularization. They discourage certain kinds of large or excessive model parameters.

Some configurations use target networks: slowly updated copies of the learning models. These make the training targets change more gradually.

Some also use early stopping: ending training when the validation-based stopping rule is triggered. Here, that rule watches the critic's validation loss; it does not directly select the highest-profit test portfolio.

Source: original_pennylane/qrl-dpo-public/ddpg/ddpg_functions.py.

8. The two learning-algorithm families

DDPG stands for Deep Deterministic Policy Gradient. In this project, it learns an actor that directly outputs allocation amounts. Its future-value calculation uses the next allocation proposed by an actor.

The project's Deep Q-Learning variant also uses an actor and critic. Its main distinction is that it samples candidate next actions, scores them, and uses the highest score in the critic's target.

This is worth specifying because the repository's implementation is more specialized than the common introductory example of Q-learning choosing between a few discrete buttons.

Both algorithm families can use ordinary neural networks or quantum models as their actor and critic.

Source: original_pennylane/qrl-dpo-public/q_learning/q_learning_functions.py.

9. What the quantum component does

The quantum component replaces the model's internal calculation. The surrounding data preparation, learning loop, and portfolio evaluation remain largely the same.

An ordinary neural network is a sequence of adjustable mathematical transformations. Training changes its internal numbers so its outputs become more useful.

The quantum version uses an adjustable quantum circuit for that role. A circuit is a sequence of operations on qubits, the basic units of quantum information.

In the current benchmark implementation, a normal computer simulates that circuit. The project is not sending the training jobs to a physical quantum computer.

The quantum model processes one market state through the following steps.

Transform the input

The main benchmarks use a function named radial_to_linear. It standardizes the input, meaning it centers and rescales the values, and creates four versions: the standardized values, their squares, their sine values, and their cosine values.

Sine and cosine are mathematical functions that turn numbers into curved, repeating patterns. Here they supply additional mathematical representations of the same input. The four-investment, ten-day example expands from 40 numbers to 160.

Encode the numbers

The model loads the numbers into a simulated quantum state using amplitude encoding. This means representing the input through the numerical coefficients that describe that state.

The code pads the input with zeros and rescales it to satisfy the state's mathematical requirements. Eight qubits have 256 such coefficients, so the 160-feature example fits into an eight-qubit state.

The implementation also requires enough qubits for its chosen number of outputs, which affects larger portfolios. This compact representation does not, by itself, demonstrate a speed advantage. Preparing the state and simulating its evolution still require work.

Apply adjustable operations

The circuit applies operations called gates. The main configuration uses RY rotations, adjustable operations on individual qubits. The rotation angles are the model parameters training changes.

When its configured circuit has multiple passes, the code inserts CNOT gates between passes. These are two-qubit operations that allow the circuit to couple qubits rather than treating every qubit independently.

Read the result

The simulator calculates numerical outputs called expectation values. An expectation value is an average measurement outcome predicted by the quantum state. Here those outputs lie between -1 and +1.

For the actor, another calculation converts the outputs into portfolio weights. For the critic, the output supplies its score.

Source: qiskit_port/qiskit_exact_amplitude_finite_diff_qnn.py.

10. Convert outputs into allocations

If short selling is disabled, the actor uses softmax, a transformation that makes allocations positive and makes them sum to one.

The main benchmarks enable short selling. They instead divide each output by the sum of all outputs.

Short selling means taking a negative position that benefits when an investment falls. An allocation of 120% to one asset and -20% to another still adds to 100%, but its total exposure is greater than the portfolio's starting value.

The evaluator also applies a negative-weight adjustment and then normalizes again.

This is not equivalent to a comprehensive real-world borrowing and exposure-control system. In particular, dividing by a sum near zero can produce very large allocations. That is relevant when interpreting extreme returns in the saved experiments.

11. How the quantum model learns

Training requires measuring how outputs respond to small changes. These sensitivities are called gradients.

A gradient tells the training process something like: If I increase this adjustable number slightly, does the training objective improve or worsen?

The Qiskit version uses two techniques:

- Parameter shift for circuit parameters: run the circuit with a rotation shifted in two specified directions and combine the results.
- Finite differences for input sensitivities: increase an input slightly, decrease it slightly, and estimate the slope from the difference.

Input sensitivities matter because the critic must tell the actor how changing its proposed allocation would affect the critic's score.

These extra evaluations explain much of the Qiskit implementation's expense. One learning update can require many circuit simulations.

PennyLane's original simulator-based implementation obtains derivatives through its simulation calculations, providing a different computational route.

12. Verify that the two implementations agree

The Qiskit port includes parity tests: checks that it reproduces the intended model calculations.

They compare both implementations using matching inputs and model parameters, checking the outputs, gradients of trainable parameters, and gradients with respect to inputs. Tests cover different encoding options, circuit configurations, and output transformations.

One conversion issue is bit ordering: the toolkits arrange the numerical representation of qubits differently. The port explicitly reorders the encoded values so the calculations correspond.

There is a distinction between matching one calculation and matching an entire training run. Very small calculation differences can accumulate through repeated updates, random exploration, and replay sampling. The repository includes comparisons showing that final trained results can differ.

Matching calculations within a tolerance supports implementation consistency; it does not establish identical trained portfolios or superior investment performance.

Source: qiskit_port/test_full_pennylane_qiskit_parity.py.

13. Test the trained strategy on later dates

After training, the system performs a backtest. A backtest applies a strategy to historical dates to calculate how it would have performed under the evaluator's assumptions.

There are two modes. Static Portfolio Optimization, or SPO, chooses one allocation using history available before the test period, then uses those same target weights throughout the test. Dynamic Portfolio Optimization, or DPO, repeatedly uses the latest available history to calculate a new allocation.

With a ten-day lookback and ten-day rebalance interval, DPO looks at the ten days preceding the next decision, produces an allocation, applies it to the following ten days, then advances and repeats.

Rebalancing means changing the allocation among investments. The trained actor makes new decisions during evaluation, but its internal model parameters are not retrained at every rebalance.

Each day's simulated portfolio return is the sum of each investment's weight multiplied by that investment's daily return. For a two-investment example:

> **60% x 2% + 40% x -1% = 0.8%.**

The evaluator assumes constant target weights inside each interval and ignores how actual holdings' percentages would drift as prices change. It is a simplified portfolio simulation rather than a complete brokerage-account simulation.

Source: qiskit_port/portfolio_metrics.py.

14. Compare eight model entries

The benchmark compares:

- Equal Weight: divide the money evenly.
- Mean-Variance Optimization: mathematically balance estimated returns against risk and how investments move together.
- Classical DDPG.
- Classical Deep Q-Learning.
- PennyLane quantum DDPG, labeled QDPG.
- PennyLane Quantum Q-Learning.
- Qiskit QDPG.
- Qiskit Quantum Q-Learning.

Classical means ordinary computation without a quantum circuit model. The simple methods are baselines: reference strategies used to determine whether a more complicated model adds value.

The first main benchmark uses four assets and 240 rows from the original dataset, a ten-day lookback, and up to 25 training epochs.

The second uses the custom ten-stock dataset, 240 return rows, and a five-day lookback. Its Qiskit models receive at most five epochs, while the other trained models receive up to 25. That reduced budget matters when comparing final results.

Sources: qiskit_port/run_real_data_benchmark.py; qiskit_port/run_custom_nonpaper_10_benchmark.py.

15. Measure performance from several angles

Annualized return expresses growth as an equivalent yearly rate. The code includes both the original mean-return-based calculation and a separate calculation based on compounded growth.

Volatility measures how much returns fluctuate. The Sharpe ratio measures return above the code's assumed reference rate, relative to volatility.

Value at Risk, or VaR, is a historical bad-day threshold. The 5% version identifies the return cutoff around the worst 5% of days. Conditional VaR, or CVaR, is the average loss in that worst tail.

Maximum drawdown is the largest fall from a previous portfolio-value peak. Turnover measures how much allocation changes when trades occur.

Transaction-cost-adjusted return is performance after the implemented trading-cost deduction. Runtime measures how long the experiment takes.

The default cost calculation uses ten basis points, or 0.1%, multiplied by its one-way turnover measure. It excludes the initial position-building cost and does not model every practical trading expense.

Annualizing a short test window can magnify unusually strong results into enormous yearly-looking numbers. Those are mathematical extrapolations, not a year of observed performance.

Sources: qiskit_port/portfolio_metrics.py; qiskit_port/run_stress_and_tail_tests.py.

16. Check whether the results are dependable

A parameter sweep varies settings such as lookback length, training epochs, and dataset size to see how the outcome changes.

Stability tests repeat training with different random seeds. A seed specifies a starting point for random-number generation. Changing it helps reveal whether apparent success depends heavily on a lucky initialization.

Reproducibility tests repeat the same setup with the same seed to check whether the result repeats.

Robustness checks include randomly shuffled training dates, untrained actors, randomly generated allocations, and repeated tests across different historical periods. These ask whether training adds useful information beyond what the market period or model's initial behavior already supplies. No single such check proves or disproves skill on its own.

Stress tests evaluate selected periods such as the 2020 crash, the 2022 declining market, and a calmer 2024 comparison period. Internal diagnostics record intermediate training quantities to investigate why training-loss curves jump around.

These tests are essential context for the results. For example, the saved robustness summary includes strong outcomes from some untrained actors and shuffled-data experiments, as well as changing rankings across time periods. That weakens any simple interpretation that a good headline number automatically demonstrates learned forecasting skill.

Sources: qiskit_port/run_parameter_sweep.py; run_stability_tests.py; run_robustness_checks.py; run_stress_and_tail_tests.py; run_internal_diagnostics.py (all in qiskit_port/). Saved findings: comparison_logs/robustness_summary.csv.

17. Save research records and figures

The comparison_logs/ folder stores tables of model performance, experiment logs, daily return and allocation histories, training-loss histories, comparison charts, and quantum-circuit diagrams and exports.

The .npz files are compressed collections of numerical arrays. The .json files store structured records. The .tex files contain material for LaTeX, a system often used to write research papers.

The benchmark runner can reuse saved model-result files to avoid repeating expensive work. That is result reuse, rather than resuming an interrupted model from its last training update.

The paper/ directory, paper outline, and technical documents turn experiment outputs into presentation and publication materials.

18. The planned real-hardware stage

Real quantum hardware is a planned next stage. The IonQ document proposes training locally, freezing the learned model parameters, and running selected decisions on hardware.

Freezing means keeping model parameters fixed. Inference means using a trained model to produce an answer without updating it.

The proposed experiment would compare hardware-generated outputs with simulator outputs and measure the effects of hardware errors and repeated measurements. The existing simulator results should not be described as completed hardware results.

Source: qiskit_port/ionq_inference_plan.md.

19. What to know about this checkout

Two details matter for anyone using the checkout reviewed for this guide. The original data/price_data.parquet.gzip file is missing from original_pennylane/qrl-dpo-public/, so scripts depending on it need that input restored before they can run. Saved results and custom datasets are present.

Also, the wrapper's fast_statevector option currently stores a backend name but does not select a separate faster implementation; the core Qiskit simulation path remains the one described above.

The most useful starting point for following an actual run is qiskit_port/run_real_data_benchmark.py. It sets the experiment configuration, loads and splits the data, and hands the work to benchmark_lib.py, which builds, trains, evaluates, and saves each model.

For deeper implementation detail, see TECHNICAL_DOCUMENTATION.md and qiskit_port/README_QISKIT_PORT.md. For the original-code reproduction, see pennylane_reproduction/README.md. All source paths in this guide are relative to the project root.